

Project Executable Files

Date	05 July 2026
Team ID	XXXXXX
Project Name	AI Learning Support Assistant
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA (CSV storage is used instead of a database)
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	No (or Yes, if you deployed it)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA (Streamlit web application)
8	Dockerfile / Containerization config (if applicable)	No (if not created)
9	Test files and test results	Yes (Epic 6 testing documentation)
10	Demo video or walkthrough (if required)	Yes (if recorded)

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```

learning-support-engine/
|
├─ app.py
├─ requirements.txt
├─ README.md
├─ .env.example
|
├─ data/
|   ├─ interaction_history.csv
|   ├─ tokenizer.pkl
|   └─ label_encoder.pkl
|
├─ models/
|   ├─ bilstm_model.keras
|   └─ bert_model/
|
├─ services/
|   ├─ bilstm_model.py
|   ├─ bert_model.py
|   ├─ gemini_service.py
|   ├─ mixed_emotion.py
|   ├─ prediction_schema.py
|   ├─ interaction_logger.py
|   ├─ history_loader.py
|   └─ analytics.py
|
├─ assets/
|
└─ documentation/

```

If your project structure differs slightly, adjust the folder names to match your repository

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not Deployed <i>(or your deployed URL if available)</i>
Login Credentials (Demo)	Not Required
Platform / Hosting Provider	Local Streamlit Application <i>(or Streamlit Community Cloud if deployed)</i>
Repository Link	<i>Paste your GitHub Repository URL</i>
Demo Video Link	<i>Paste your demo video link if available</i>

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Prerequisites

- Python 3.11 or above
- Git
- Internet connection (required for Google Gemini API)
- Google Gemini API Key
- Visual Studio Code (recommended)

Step 1

Clone the repository

```
git clone <your-github-repository-url>
```

Step 2

Navigate to the project directory

```
cd learning-support-engine
```

Step 3

Create a virtual environment

```
python -m venv venv
```

Step 4

Activate the virtual environment

Windows

```
venv\Scripts\activate
```

Linux / macOS

```
source venv/bin/activate
```

Step 5

Install dependencies

```
pip install -r requirements.txt
```

Step 6

Configure environment variables

Create a `.env` file and add your Google Gemini API key.

Example:

```
GOOGLE_API_KEY=your_api_key_here
```

Step 7

Run the application

```
streamlit run app.py
```

Step 8

Open the browser

<http://localhost:8501>

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	First prediction may take slightly longer due to model loading	Resolved using <code>@st.cache_resource</code> to cache models
2	Google Gemini responses depend on internet connectivity	Ensure a stable internet connection and a valid API key
3	Interaction history is stored in a CSV file instead of a database	Acceptable for the current project scope; suitable for small-scale usage